
Fountain View Hall

Release 0.2.0

Derek R. Neilson

Jun 07, 2026

CONTENTS:

1	User Guide	3
1.1	Installation	3
1.1.1	User Prerequisites	3
1.1.2	Minimal Getting Started	3
1.1.3	Activate Completion by Operating System and Shell	4
1.2	Developer Installation	4
2	References	5
3	Design	7
3.1	Fountain View Hall Pseudocode and User Journeys	7
3.1.1	Problem Statement	7
3.2	Overview for Fountain View Hall	7
3.2.1	Overview	7
3.2.1.1	Program Requirements	7
3.2.1.2	Business Rules	7
3.2.2	Planned Technology Usage	8
3.2.2.1	Editing/Developer Tools	8
3.2.2.2	Python Packages	8
3.3	References Used to Make This Project	8
3.4	Style Guide	8
3.4.1	Scope of Guide	8
	Python Module Index	9
	Index	11

Fountain View Hall's management tool, `fvh-manager` is used to manage reservations at Fountain View Hall.

USER GUIDE

This guide will walk you through `fvh-manager` and covers topics such as how to install and use `fvh-manager`.

1.1 Installation

1.1.1 User Prerequisites

To verify you meet all prerequisites, run:

```
uv --version
```

Expected output:

```
uv 0.x.x
```

Not:

```
command not found: uv
```

Important

Make sure the `uv` command runs before running the program.

If `uv --version` fails, use [this guide](#) to install `uv`.

1.1.2 Minimal Getting Started

Make sure you run these commands from the project directory

Run this once:

```
uv sync
```

Then run this to start on every subsequent run:

```
uv run fvh-manager
```

1.1.3 Activate Completion by Operating System and Shell

If desired, you can activate shell completions by running one of the following commands, depending on your operating system and shell:

Operating system	Shell	Activation command
Linux/macOS	Bash	<code>source completions/fvh-manager.bash</code>
Linux/macOS	Zsh	<code>source completions/_fvh-manager</code>
Linux/macOS	Fish	<code>source completions/fvh-manager.fish</code>
Windows	PowerShell	<code>. .\completions\fvh-manager.ps1</code>

Once shell completions are activated, make sure your virtual environment is active. You can then run the program with:

```
fvh-manager
```

To view available completions, type:

```
fvh-manager <tab>
```

1.2 Developer Installation

Meet all previous *prerequisites* and additional ones as specified bellow.

Install developer dependencies for python:

```
uv sync --all-groups
```

Verify that `pdflatex` is installed:

```
pdflatex -v
```

Verify that `git` is installed:

```
git -v
```

Verify that `lefthook` is installed:

```
lefthook -v
```

REFERENCES

Command-line entry point for the project.

author

Derek R. Neilson

This module defines the project's Typer application and should remain lightweight by design.

Typer is a command-line interface (CLI) framework used throughout this project. It provides decorators such as `typer.command()` for registering CLI commands.

See also

[Typer documentation](#)

`src.main.create_engine_command(in_memory=False)`

Create a SQLite database engine.

Creates either an in-memory SQLite database or, by default, a persistent database at `DB_PATH`, which points to `database/data.db`.

Parameters

`in_memory` (bool) – If `True`, create an in-memory database. If `False`, create a persistent database at `DB_PATH`.

Returns

A SQLAlchemy engine representing the configured database connection.

Return type

Engine

The SQLAlchemy database URL has the general form:

```
dialect[+driver]://user:password@host/dbname[?key=value...]
```

The default project database URL has the form:

```
sqlite+pysqlite:///database/data.db
```

If `in_memory` is `True`, the database URL has the form:

```
sqlite+pysqlite:///memory:
```

See also

[SQLAlchemy Engine documentation](#) [Typer documentation](#)

The design process has been documented quite thoroughly, but not all decisions are documented.

3.1 Fountain View Hall Pseudocode and User Journeys

3.1.1 Problem Statement

The project must meet all *requirements*.

3.2 Overview for Fountain View Hall

3.2.1 Overview

Fountain View Hall wants a program that tracks event attendance and projected revenue for multiple bookings.

3.2.1.1 Program Requirements

- Repeatedly prompt for event name, guest count, and estimated revenue
- Continue until the user enters “done”
- Track total events processed
- Track total guests
- Track total projected revenue
- Calculate the average guests per event
- Determine the largest event

3.2.1.2 Business Rules

- Pricing Rules
 - Large events receive a discount
- Operational Rules
 - Every booking must include a guest count
 - Booking total must be calculated before the discount is applied
 - Revenue totals should be tracked

3.2.2 Planned Technology Usage

I intended to use various technologies throughout this project; some are very necessary, and others save time. Each technology will be categorized and described below:

3.2.2.1 Editing/Developer Tools

- *Git* and *GitHub* as a VCS (version control system).¹
- *uv* for package, project, and environment management.
- *lefthook* as a Git hook manager.
- *pre-commit* to run pre-commit scripts.²
 - *ruff* for Python linting and formatting running as a pre-commit script.
- *Sphinx* to help build this beautiful documentation.
- *pytest* as a testing suite.

3.2.2.2 Python Packages

- *typer* for command line interface completions and argument parsing.
 - *SQLite3* as a database³
 - *SQLAlchemy* for object-relational mapping (ORM)
-

3.3 References Used to Make This Project

ReStructuredText guides were used as references for this project (see footnote).¹

Note

I had zero RST experience before reading these guides; they were very helpful.

The [Typer home page](#) was used for learning about Typer and was integral to getting the CLI up.

[SQLAlchemy](#) is another tool I never used before. Its own [documentation](#) was sufficient for my usage.

3.4 Style Guide

This style guide is for me and serves as a reminder to me.

3.4.1 Scope of Guide

Seeing as I am the sole developer of this project, I don't think it is necessary to define all stylings. That being said, I will define some style choices.

¹ Go to the [project repo](#) to see commit history.

² All scripts (in the scripts directory) are written for Unix systems and use the bash language.

³ See [SQLite's home page](#) for more details.

¹ reStructuredText guides: reStructuredText Markup Specification, reStructuredText Primer, reStructuredText Primer by Material for Sphinx, and reStructuredText Primer by Sphinx Note: they all are missing some info that others have, except for reStructuredText Markup Specification which is very dense and lacks rendered examples.

PYTHON MODULE INDEX

S

`src.main`, 5

INDEX

C

`create_engine_command()` (*in module src.main*), 5

M

`module`
 `src.main`, 5

S

`src.main`
 `module`, 5